Manual Técnico

Sistema Integral Web de Control de Producción y Distribución de Alimentos Congelados — Distribuidora Yaneth

Apéndice A — Arquitectura, instalación, configuración, respaldos y mantenimiento

Versión 2.0 — Octubre 2026

Elaborado por: Dennis Cornel

Contenido

1. Introducción y alcance 4
2. Arquitectura del sistema 4
2.1 Visión general 4
2.2 Recorrido de una petición 5
2.3 Tecnologías y versiones 6
2.4 Estructura del código 6
2.5 Seguridad y control de acceso 7
2.6 Documentación de la API 8
3. Requisitos y topología de despliegue 8
3.1 Topología de despliegue 8
3.2 Hardware y software de los servidores 11
3.3 Requisitos de los equipos cliente 11
3.4 Accesos y datos necesarios antes de instalar 11
4. Variables de entorno 12
4.1 Backend (archivo Backend/.env) 12
4.2 Frontend (archivo Frontend/.env.production) 13
5. Instalación en producción 13
5.1 Día 1 – Preparación de los servidores 13
5.2 Día 2 – Migración y configuración 15
5.3 Día 3 – Configuración inicial de datos y pruebas 18
6. Operación de los servicios 19
6.1 Orden de arranque 19
6.2 Comandos de operación 19
6.3 Registros (logs) 20
7. Respaldos y restauración 20
7.1 Qué se respalda 20
7.2 Respaldo diario automático 20
7.3 Restauración 21
7.4 Prueba periódica de restauración 21
8. Mantenimiento 21

8.1 Rutinas de mantenimiento 21
8.2 Actualizar a una nueva versión del sistema 22
8.3 Migraciones de la base de datos 22
8.4 Gestión de usuarios y accesos 22
8.5 Parámetros operativos 23
8.6 Solución de problemas 23
8.7 Buenas prácticas de seguridad 24
Anexo A. Checklist de aceptación de la instalación 24
Anexo B. Instalación local para desarrollo 25
B.1 Requisitos 25
B.2 Base de datos y Redis 25
B.3 Backend 25
B.4 Frontend 26
B.5 Diferencias con producción 26

1. Introducción y alcance

Este manual describe cómo está construido el Sistema Integral Web de Control de Producción y Distribución de Alimentos Congelados, cómo se instala en los servidores de Distribuidora Yaneth y cómo se opera, respalda y mantiene una vez en producción. Está dirigido a la persona encargada del servidor y a cualquier desarrollador que dé continuidad al sistema.

Forma parte del paquete de apéndices que se entrega junto con el código fuente:

Apéndice	Contenido	Ubicación en el repositorio del backend
A	Manual técnico (este documento)	docs/A_Manual_Tecnico/
B	Manual de usuario, por rol y por módulo	docs/B_Manual_de_Usuario/
C	Esquema de la base de datos y diagrama entidad-relación	docs/C_Esquema_BD/ (remite a sql/ y diagramas/base_datos/)
D	Código fuente	Repositorios distri_yaneth_backend y distri_yaneth_frontend

El procedimiento de instalación (capítulo 5) corresponde al plan de tres jornadas resumido en el Capítulo V de la tesis; aquí se detallan los comandos de cada actividad. La sección 3.1 describe la topología de despliegue (Nginx y los servidores de aplicación, base de datos y colas, en un mismo equipo o separados) y el Anexo B explica cómo preparar un equipo de desarrollo.

2. Arquitectura del sistema

2.1 Visión general

El sistema es una aplicación web de tres capas con procesamiento asíncrono de las operaciones de base de datos. Se compone de cuatro piezas:

- **Cliente web (React + Vite).** Interfaz que usa el personal desde el navegador. Se compila a archivos estáticos (HTML, CSS y JavaScript) que entrega Nginx, el servidor web del sistema.
- **API REST (Node.js + Express 5).** Recibe las peticiones del cliente, valida el token de sesión (JWT) y el rol del usuario, y encola cada operación de base de datos.
- **Cola de trabajos (BullMQ sobre Redis).** Desacopla la petición HTTP de su procesamiento y limita cuántas operaciones se ejecutan en paralelo contra la base de datos.
- **Worker (Node.js).** Toma los trabajos de la cola y ejecuta las consultas y transacciones contra MySQL/MariaDB. Corre dentro del mismo proceso que la API: al iniciar la API se inicia también el worker.

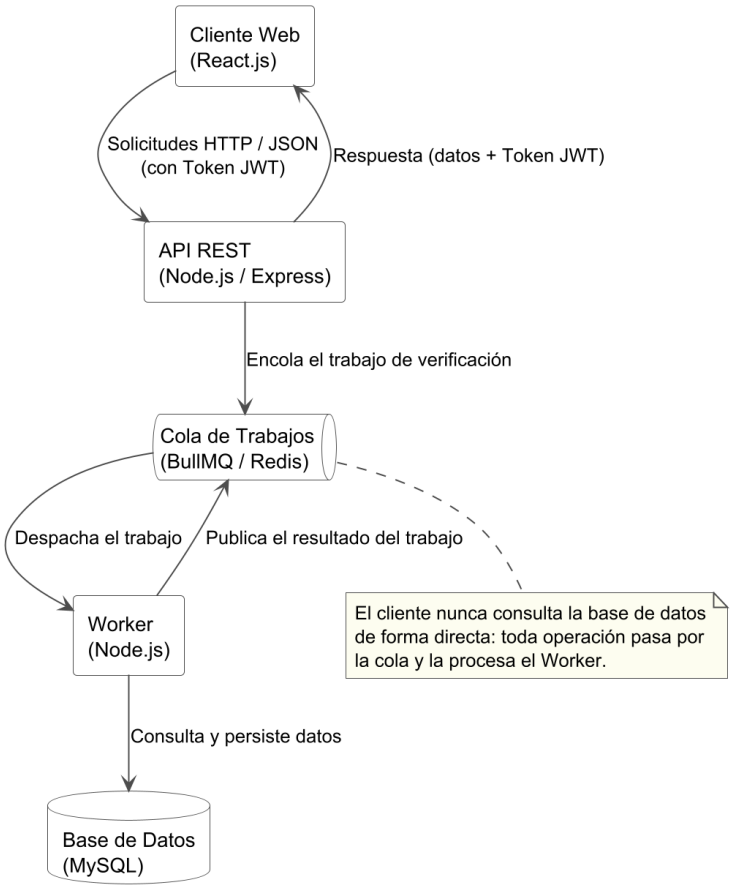


Figura 1. Arquitectura de componentes del sistema.

El cliente nunca consulta la base de datos de forma directa: toda operación pasa por la API, la cola y el worker. Redis solo guarda trabajos en tránsito (se eliminan 60 segundos después de completarse); toda la información del negocio vive en la base de datos.

Estas piezas pueden ejecutarse en un solo servidor o repartirse entre un servidor de aplicación (Nginx, API y worker), uno de base de datos y uno de colas; la sección 3.1 describe ambas variantes.

2.2 Recorrido de una petición

1. El usuario inicia sesión en `POST /api/users/login`. La API valida la contraseña (bcrypt) y devuelve un token JWT con el identificador, nombre y rol del usuario, con vigencia definida por `JWT_EXPIRES_IN` (8 h por defecto).
2. El cliente envía el token en la cabecera `Authorization: Bearer <token>` en cada petición posterior.
3. El middleware `authenticateToken` verifica la firma y la vigencia del token; `authorizeRoles` comprueba que el rol tenga acceso a la ruta solicitada (respuestas 401 o 403 si no).

4. El controlador valida los datos de entrada y llama al modelo, que encola un trabajo en BullMQ (`enqueueDbJob`) y espera su resultado hasta 60 segundos.
5. El worker ejecuta el manejador del dominio correspondiente (`src/workers/domains/*.worker.js`). Las operaciones que modifican varias tablas se ejecutan en una transacción y registran el cambio en la tabla `auditoria_cambios`.
6. El resultado vuelve por la cola a la API, que responde al cliente en formato JSON.

2.3 Tecnologías y versiones

Tabla 1. Tecnologías del sistema (package.json de cada repositorio) y del servidor.

Capa	Tecnología	Versión declarada
Backend	Node.js	20.19 o superior (recomendado 22 LTS)
Backend	Express	5.2
Backend	BullMQ / ioredis	5.76 / 5.10
Backend	mysql2 (pool de conexiones)	3.22
Backend	jsonwebtoken / bcryptjs	9.0 / 3.0
Backend	swagger-jsdoc / swagger-ui-express	6.3 / 5.0
Backend	morgan (registro de peticiones), cors, dotenv	1.10 / 2.8 / 17.4
Frontend	React / React DOM	19.2
Frontend	Vite (compilación)	8.0
Frontend	Recharts (gráficas del dashboard)	3.10
Frontend	jsPDF + jspdf-autotable / ExcelJS (exportaciones)	4.2 + 5.0 / 4.4
Frontend	SweetAlert2 (notificaciones)	11.26
Datos	MySQL 8.0 (paquete de Ubuntu 24.04)	Motor InnoDB, juego de caracteres utf8mb4
Cola	Redis (paquete de Ubuntu 24.04)	7.0 (mínimo 6.2)
Servidor	Ubuntu Server	24.04 LTS
Servidor	Nginx (servidor web y proxy inverso) + Certbot	1.24
Servidor	PM2 (gestor de procesos de la API)	5.x

2.4 Estructura del código

El código está dividido en dos repositorios independientes:

Tabla 2. Estructura del repositorio del backend.

Carpeta (backend)	Contenido
src/server.js, src/app.js	Arranque: conexión a la base de datos, cola y worker; registro de rutas y permisos por rol.
src/config/	Pool de MySQL (db.js), conexión a Redis (redis.js) y especificación Swagger (swagger.js).
src/middlewares/	auth.middleware.js: validación del token JWT y autorización por rol.
src/routes/	Definición de endpoints por módulo, con su documentación Swagger.
src/controllers/	Validación de entrada y armado de la respuesta HTTP.
src/models/	Encolan los trabajos de base de datos de cada módulo.
src/queues/	db.queue.js: cola BullMQ y espera de resultados.
src/workers/	db.worker.js (registro de manejadores), domains/ (lógica y SQL por

	módulo) y shared/ (constantes, auditoría, códigos de fase).
sql/schema.sql	Esquema consolidado de la base de datos (34 tablas).
sql/migrations/	Cambios incrementales fechados, para actualizar bases ya existentes.
diagramas/	Fuentes PlantUML y PNG de los diagramas (incluye el diagrama entidad-relación).
scripts/	Utilidades de desarrollo (siembra de datos de demostración). No se ejecutan en producción.
docs/	Apéndices A, B y C.

Tabla 3. Estructura del repositorio del frontend.

Carpeta (frontend)	Contenido
src/pages/	Página de inicio de sesión y contenedor principal (Dashboard).
src/components/	Menú lateral, widgets y gráficas del dashboard, componentes comunes.
src/features/	Un módulo por carpeta (entries, maturation, production, orders, routes, returns, traceability, etc.).
src/services/	Llamadas HTTP a la API; todas usan la variable VITE_API_URL.
src/config/rolePermissions.js	Módulos visibles por rol y agrupación del menú lateral.
src/utils/	Formato de códigos de fase, notificaciones y utilidades de exportación.

2.5 Seguridad y control de acceso

- Contraseñas almacenadas con hash bcrypt (10 rondas).
- Sesión sin estado mediante JWT firmado con `JWT_SECRET`; al vencer, el usuario debe iniciar sesión de nuevo.
- Cuatro roles: Administrador, Producción, Logística y Piloto. El menú del cliente muestra solo los módulos del rol, y la API vuelve a validar el rol en cada ruta (la restricción no depende solo de la interfaz).
- Borrado lógico: los registros se marcan como Inactivos en lugar de eliminarse, lo que preserva la trazabilidad.
- Auditoría: cada alta, modificación o baja queda en `auditoria_cambios` con el usuario, la fecha y los valores antes y después del cambio.

Tabla 4. Permisos por prefijo de la API (src/app.js).

Prefijo de la API	Roles con acceso
/api/users (gestión de usuarios)	Administrador (el inicio de sesión es público)
/api/proveedores, /api/productos, /api/inventario, /api/entradas-mercancia, /api/maduracion, /api/redes-verdes, /api/trazabilidad	Administrador, Producción, Logística
/api/produccion	Administrador, Producción
/api/clientes, /api/tipos-servicio, /api/servicios-vehiculo	Administrador, Logística
/api/pedidos, /api/rutas, /api/reporte-kilometraje-vehiculo	Administrador, Logística, Piloto
/api/devoluciones	Administrador, Logística, Piloto, Producción (recepción: Administrador y Logística; resolución: además Producción)
/api/vehiculos	Consulta: Administrador, Logística, Piloto; altas y cambios: Administrador, Logística

/api/auditoria	Administrador

2.6 Documentación de la API

La API publica su documentación interactiva (OpenAPI/Swagger) y un punto de verificación de estado:

Ruta	Uso
GET /health	Verifica que la API esté en línea. Responde {"ok": true, "message": "API funcionando"}.
GET /api-docs	Documentación interactiva de todos los endpoints (Swagger UI).
GET /api-docs.json	Especificación OpenAPI en formato JSON.

3. Requisitos y topología de despliegue

3.1 Topología de despliegue

El sistema se publica en servidores con Ubuntu Server 24.04 LTS. La instalación distingue tres funciones, que pueden estar en un solo equipo o repartirse entre varios:

- Servidor de aplicación (obligatorio). Ejecuta Nginx, que entrega el frontend compilado y reenvía las rutas /api a la API, y la API con su worker (Node.js administrado por PM2). Es el único servidor que recibe tráfico de los usuarios (puertos 80 y 443).
- Servidor de base de datos. Ejecuta MySQL 8, donde vive toda la información del negocio. Puede ser el mismo servidor de aplicación o un equipo dedicado.
- Servidor de colas. Ejecuta Redis, que BullMQ usa para la cola de trabajos. Puede ser el mismo servidor de aplicación, el de base de datos o un equipo dedicado.

La elección no cambia el código: la API localiza MySQL y Redis con las variables DB_HOST y REDIS_HOST (capítulo 4). Por eso se documentan dos variantes, y los pasos de instalación indican en qué servidor se ejecuta cada actividad.

Tabla 5. Variantes de despliegue.

Variante	Cuándo conviene	Qué cambia en la configuración
A. Todo en uno	Un solo equipo disponible para el sistema. Es la configuración recomendada para la instalación inicial de Distribuidora Yaneth.	MySQL y Redis escuchan solo en 127.0.0.1. DB_HOST y REDIS_HOST = 127.0.0.1.
B. Servidores separados	La base de datos necesita un equipo propio (más memoria, discos o respaldos independientes) o se quiere aislar la carga. Puede separarse solo MySQL, solo Redis o ambos.	MySQL y Redis escuchan en la IP privada de su servidor y solo aceptan conexiones del servidor de aplicación (IP-APP). Redis exige contraseña. DB_HOST = IP-DB y REDIS_HOST = IP-REDIS.

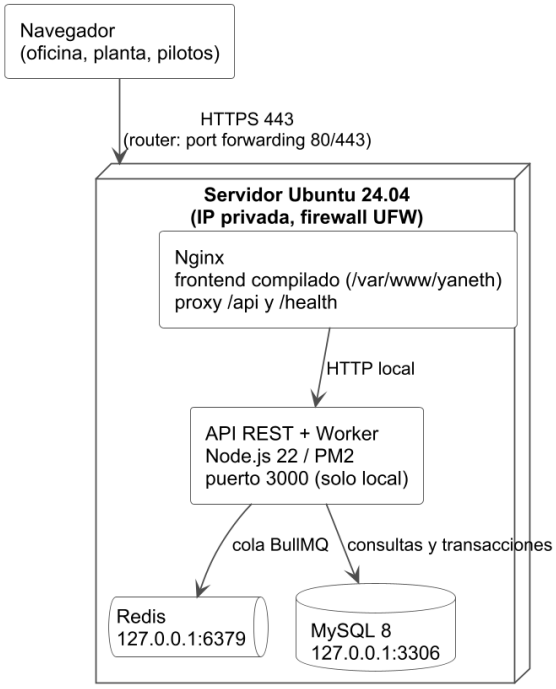


Figura 2. Variante A: Nginx, API, MySQL y Redis en un solo servidor.

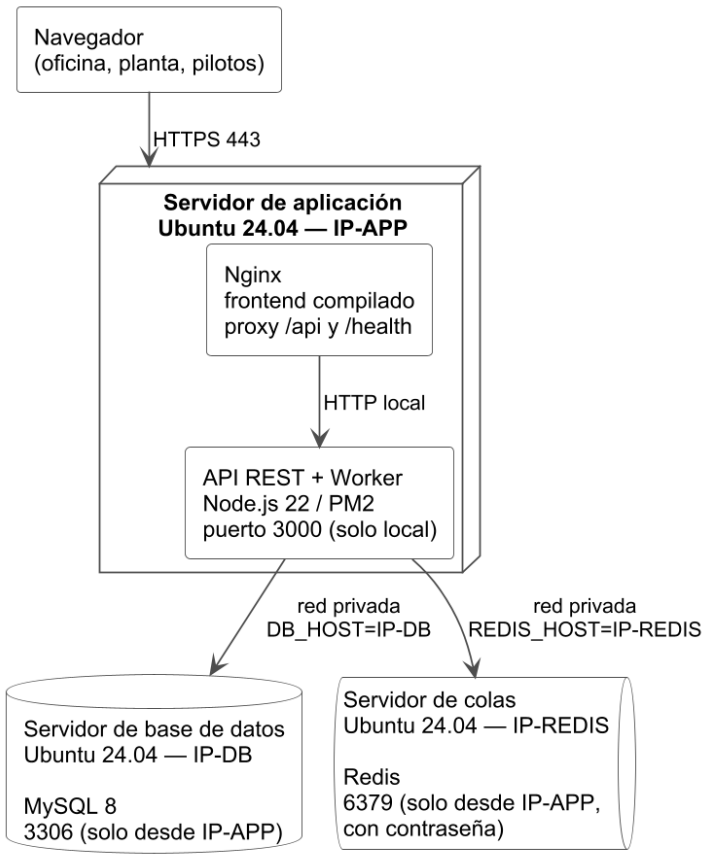


Figura 3. Variante B: MySQL y Redis en servidores propios dentro de la red privada.

En este documento IP-APP, IP-DB e IP-REDIS representan las IP privadas fijas de cada servidor. En la variante A las tres son el mismo equipo y, en los comandos, IP-DB e IP-REDIS se sustituyen por 127.0.0.1. Los pasos marcados «Solo variante B» se omiten en un servidor único.

Importante: las conexiones de la API hacia MySQL y Redis no van cifradas (el sistema no configura TLS hacia esos servicios). En la variante B los servidores deben comunicarse por una red privada (la LAN de la empresa o la red privada del proveedor de nube) y los puertos 3306 y 6379 nunca se publican en Internet.

Tabla 6. Puertos y flujos de red.

Origen → destino	Puerto	Regla
Usuarios (Internet o LAN) → Nginx	80 y 443/TCP	Abierto. El router reenvía (port forwarding) 80 y 443 hacia IP-APP.
Administrador → cualquier servidor	22/TCP (SSH)	Solo desde la red interna.
Nginx → API	3000/TCP	Solo local, dentro del servidor de aplicación. Nunca se abre.
API → MySQL	3306/TCP	Variante A: local. Variante B: solo desde IP-APP.

Origen → destino	Puerto	Regla
API → Redis	6379/TCP	Variante A: local. Variante B: solo desde IP-APP, con contraseña.

3.2 Hardware y software de los servidores

Tabla 7. Requisitos de cada servidor.

Componente	Requisito mínimo	Requisito recomendado
Sistema operativo (todos)	Ubuntu Server 24.04 LTS	Ubuntu Server 24.04 LTS con actualizaciones de seguridad automáticas
Servidor de aplicación (en la variante A también aloja MySQL y Redis)	2 vCPU, 4 GB RAM, 40 GB SSD	4 vCPU, 8 GB RAM, 80 GB SSD
Servidor de base de datos (variante B)	2 vCPU, 4 GB RAM, 50 GB de almacenamiento	Instancia dedicada con disco adicional para respaldos
Servidor de colas (variante B)	1 vCPU, 1 GB RAM dedicado	2 GB RAM dedicado, persistencia AOF
Software del servidor de aplicación	Node.js 20.19, Nginx, PM2, Git	Node.js 22 LTS, Nginx 1.24 (paquete de Ubuntu), PM2 5
Software de datos	MySQL 8.0, Redis 7.0	MySQL 8.0 y Redis 7.0 (paquetes de Ubuntu 24.04)
Red	IP privada fija por servidor; dominio o IP pública con port forwarding 80/443 hacia IP-APP	Dominio propio con HTTPS; enlace redundante o plan de datos de respaldo

3.3 Requisitos de los equipos cliente

- Navegador actualizado: Google Chrome, Microsoft Edge o Mozilla Firefox.
- Pantalla de al menos 1366 × 768 en oficina y planta. Los pilotos pueden usar un teléfono o tableta: la interfaz se adapta al ancho de pantalla.
- Acceso de red al dominio del sistema.

3.4 Accesos y datos necesarios antes de instalar

- Usuario con permisos sudo y acceso SSH en cada servidor.
- Variante de despliegue elegida (A o B) y la IP privada fija de cada servidor (IP-APP, IP-DB, IP-REDIS).
- Dominio del sistema apuntando a la IP pública de la empresa, y acceso al router para reenviar los puertos 80 y 443 hacia IP-APP. El dominio es necesario para el certificado HTTPS gratuito de Let's Encrypt.
- Contraseñas que se definirán: usuario de aplicación de MySQL (app_yaneth), usuario de respaldos (respaldo), contraseña de Redis y secreto JWT.
- Lista real del personal (nombre completo, usuario y rol) para el alta de usuarios.

- Catálogos base de la empresa: clientes, proveedores, vehículos, productos por tipo, tipos de etapa (Pelado, Corte, Fritura y Embalaje), tipos de merma, cuartos fríos y el umbral de Grados Brix vigente.

4. Variables de entorno

4.1 Backend (archivo Backend/.env)

Se crean copiando la plantilla .env.example en el servidor de aplicación. Los valores de la plantilla son de ejemplo y nunca se usan en producción.

Tabla 8. Variables de entorno del backend.

Variable	Descripción	Valor por defecto
PORT	Puerto en el que escucha la API. Nginx la alcanza localmente; no se abre en el firewall.	3000
DB_HOST / DB_PORT	Dirección de MySQL: 127.0.0.1 si está en el servidor de aplicación (variante A) o IP-DB (variante B).	localhost / 3306
DB_USER / DB_PASSWORD	Usuario de aplicación app_yaneth y su contraseña (no usar root).	root / (vacío)
DB_NAME	Nombre de la base de datos.	proyecto_graduacion
REDIS_HOST / REDIS_PORT	Dirección de Redis: 127.0.0.1 (variante A) o IP-REDIS (variante B).	127.0.0.1 / 6379
REDIS_PASSWORD	Contraseña definida en requirepass. Obligatoria en la variante B y recomendada en la A.	(vacío)
REDIS_USERNAME	Solo si se usan usuarios ACL de Redis; con requirepass se deja vacío.	(vacío)
REDIS_DB	Número de base lógica de Redis usada por la cola.	0
DB_QUEUE_NAME	Nombre de la cola BullMQ. Debe ser distinto si dos instancias comparten Redis.	db-queries
DB_QUEUE_CONCURRENCY	Trabajos que el worker procesa en paralelo.	5
MATURATION_BRIX_THRESHOLD	Grados Brix a partir de los cuales un control de maduración pasa el sub-lote a "Listo para producción".	22
JWT_SECRET	Secreto de firma de los tokens. Generar uno largo y aleatorio y mantenerlo privado.	dev-secret-change-me (inseguro)
JWT_EXPIRES_IN	Vigencia de la sesión.	8h
DB_*_TABLE	Nombres de tablas (p. ej. DB_USERS_TABLE). Existen por compatibilidad; no deben modificarse.	Nombres del esquema

Para generar un secreto seguro:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

4.2 Frontend (archivo Frontend/.env.production)

Variable	Descripción	Ejemplo
VITE_API_URL	URL base de la API. Es el mismo dominio que publica Nginx, sin /api y sin puerto. Se incrusta en los archivos compilados: si cambia, hay que volver a ejecutar npm run build.	https://sistema.yaneth.com

Nota: el cliente arma las rutas como VITE_API_URL + "/api/…" y Nginx reenvía /api a la API (sección 5.2, actividad 2.7). La variable no debe quedar vacía: el cliente usaría http://localhost:3000. Un archivo .env.local tiene prioridad sobre .env.production, por lo que no debe copiarse al servidor.

5. Instalación en producción

La instalación limpia se organiza en tres jornadas de aproximadamente seis horas, con márgenes para imprevistos. Los comandos corresponden a Ubuntu Server 24.04 LTS, y cada actividad indica el servidor donde se ejecuta según la topología elegida (sección 3.1).

Día	Enfoque	Duración
Día 1	Preparación de los servidores	6 h
Día 2	Migración y configuración	6 h
Día 3	Configuración inicial de datos y pruebas	6 h

5.1 Día 1 – Preparación de los servidores

1.1 Actualizar el sistema operativo — todos los servidores (30 min)

En cada servidor se aplican las actualizaciones, se fija la zona horaria (la hora de los registros y de los respaldos debe coincidir con la de la planta) y se anota su IP privada.

```
sudo apt update && sudo apt upgrade -y
sudo timedatectl set-timezone America/Guatemala
ip -4 addr        # anotar la IP privada: IP-APP, IP-DB o IP-REDIS
```

1.2 Instalar y asegurar MySQL — servidor de base de datos (45 min)

```
sudo apt install -y mysql-server
sudo systemctl enable mysql --now
sudo mysql_secure_installation
```

Se crea la base de datos y un usuario de aplicación con permisos solo sobre ella; el backend no usa la cuenta root. La parte del usuario después de @ indica desde dónde puede conectarse: localhost en la variante A y la IP del servidor de aplicación en la variante B.

```
sudo mysql
CREATE DATABASE proyecto_graduacion CHARACTER SET utf8mb4 COLLATE
utf8mb4_general_ci;
```

```
-- Variante A (MySQL en el servidor de aplicación)
CREATE USER 'app_yaneth'@'localhost' IDENTIFIED BY '<contraseña-segura>';
GRANT ALL PRIVILEGES ON proyecto_graduacion.* TO 'app_yaneth'@'localhost';

-- Variante B (MySQL en su propio servidor), en lugar de las dos líneas anteriores
CREATE USER 'app_yaneth'@'<IP-APP>' IDENTIFIED BY '<contraseña-segura>';
GRANT ALL PRIVILEGES ON proyecto_graduacion.* TO 'app_yaneth'@'<IP-APP>';

FLUSH PRIVILEGES;
EXIT;
```

Solo variante B. Por defecto MySQL escucha únicamente en 127.0.0.1; se le indica la IP privada del servidor para que el servidor de aplicación pueda alcanzarlo:

```
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
#   bind-address = <IP-DB>        (en lugar de 127.0.0.1)
sudo systemctl restart mysql
sudo ss -tlnp | grep 3306        # debe mostrar <IP-DB>:3306
```

1.3 Instalar y asegurar Redis — servidor de colas (45 min)

```
sudo apt install -y redis-server
sudo nano /etc/redis/redis.conf
#   requirepass <contraseña-redis>   (obligatorio en B, recomendado en A)
#   appendonly yes                   (persistencia AOF)
#   maxmemory-policy noeviction      (por defecto; BullMQ lo requiere)
#   bind 127.0.0.1 <IP-REDIS>        (solo variante B)
sudo systemctl enable redis-server
sudo systemctl restart redis-server
redis-cli -a '<contraseña-redis>' ping   # debe responder PONG
```

1.4 Instalar Node.js, Nginx y PM2 — servidor de aplicación (30 min)

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs nginx git
sudo npm install -g pm2
node -v && npm -v && nginx -v

# Solo variante B: clientes para probar MySQL y Redis remotos
sudo apt install -y mysql-client redis-tools
```

1.5 Configurar el firewall — todos los servidores (30 min)

Cada servidor acepta solo lo indispensable. El puerto 3000 de la API nunca se abre: Nginx la alcanza dentro del mismo equipo. En el router solo se reenvían 80 y 443 hacia IP-APP.

```
# Servidor de aplicación (único que atiende a los usuarios)
sudo ufw allow OpenSSH
sudo ufw allow 'Nginx Full'        # 80 y 443
sudo ufw enable

# Servidor de base de datos (solo variante B)
```

```
sudo ufw allow OpenSSH
sudo ufw allow from <IP-APP> to any port 3306 proto tcp
sudo ufw enable

# Servidor de colas (solo variante B)
sudo ufw allow OpenSSH
sudo ufw allow from <IP-APP> to any port 6379 proto tcp
sudo ufw enable

sudo ufw status numbered           # revisar en cada servidor
```

1.6 Copiar el código fuente — servidor de aplicación (30 min)

El código queda en /opt/yaneth-sistema, propiedad del usuario que ejecutará PM2 (no root).

```
sudo mkdir -p /opt/yaneth-sistema && sudo chown $USER:$USER /opt/yaneth-sistema
cd /opt/yaneth-sistema
git clone https://github.com/DecosUniversity/distri_yaneth_backend.git Backend
git clone https://github.com/DecosUniversity/distri_yaneth_frontend.git Frontend
```

1.7 Programar los respaldos automáticos — servidor de base de datos (45 min)

El respaldo se programa en el servidor donde está MySQL. Ver el procedimiento completo en el capítulo 7.

1.8 Probar la conectividad entre servidores — servidor de aplicación (60 min)

Desde el servidor de aplicación se comprueba que MySQL y Redis responden con las credenciales definidas (en la variante A, IP-DB e IP-REDIS son 127.0.0.1):

```
mysql -h <IP-DB> -u app_yaneth -p -e "SELECT VERSION();"
redis-cli -h <IP-REDIS> -a '<contraseña-redis>' ping      # PONG
```

Registrar en un documento interno las IP, credenciales, rutas y versiones instaladas. Margen para imprevistos: 45 min.

5.2 Día 2 – Migración y configuración

Todas las actividades del día se ejecutan en el servidor de aplicación.

2.1 Crear el esquema de la base de datos (30 min)

```
cd /opt/yaneth-sistema/Backend
mysql -h <IP-DB> -u app_yaneth -p proyecto_graduacion < sql/schema.sql
```

Importante: schema.sql elimina y vuelve a crear cada tabla (DROP TABLE IF EXISTS). Se ejecuta una sola vez sobre la base vacía; nunca sobre una base con datos. Para actualizar una base existente se usan las migraciones (sección 8.3).

2.2 Verificar las tablas (30 min)

```
mysql -h <IP-DB> -u app_yaneth -p -e "SELECT COUNT(*) FROM
information_schema.tables
  WHERE table_schema = 'proyecto_graduacion';"      # debe devolver 34
```

2.3 Crear el usuario administrador inicial (15 min)

El esquema no incluye usuarios. Se genera el hash bcrypt de una contraseña temporal y se inserta el primer Administrador; los demás usuarios se crean desde el módulo Usuarios.

```
cd /opt/yaneth-sistema/Backend && npm ci --omit=dev
node -e "console.log(require('bcryptjs').hashSync('<contraseña-temporal>', 10))"

mysql -h <IP-DB> -u app_yaneth -p proyecto_graduacion
INSERT INTO usuarios (nombre_completo, username, password_hash, rol)
VALUES ('Administrador del sistema', 'admin', '<hash-generado>', 'Administrador');
```

2.4 Configurar las variables de entorno (30 min)

```
cp .env.example .env
nano .env
chmod 600 .env
```

Valores de producción (capítulo 4). El ejemplo corresponde a la variante B; en la variante A, DB_HOST y REDIS_HOST son 127.0.0.1. Las líneas DB_*_TABLE de la plantilla se dejan como están.

```
PORT=3000
DB_HOST=<IP-DB>
DB_PORT=3306
DB_USER=app_yaneth
DB_PASSWORD=<contraseña-segura>
DB_NAME=proyecto_graduacion
REDIS_HOST=<IP-REDIS>
REDIS_PORT=6379
REDIS_USERNAME=
REDIS_PASSWORD=<contraseña-redis>
REDIS_DB=0
DB_QUEUE_NAME=db-queries
DB_QUEUE_CONCURRENCY=5
MATURATION_BRIX_THRESHOLD=22
JWT_SECRET=<secreto-generado>      # ver sección 4.1
JWT_EXPIRES_IN=8h
```

2.5 Publicar la API como servicio con PM2 (45 min)

PM2 mantiene la API (con su worker) en ejecución, la reinicia ante una falla y la levanta al reiniciar el servidor. Se inicia desde la carpeta Backend porque la API lee el archivo .env del directorio de trabajo. La espera creciente entre reintentos evita que PM2 abandone el proceso si MySQL o Redis tardan en estar disponibles.

```
cd /opt/yaneth-sistema/Backend
pm2 start src/server.js --name yaneth-api --exp-backoff-restart-delay=2000
pm2 logs yaneth-api --lines 20   # "Conexion a MySQL establecida"
pm2 save
pm2 startup systemd              # ejecutar el comando sudo que imprime
```

2.6 Compilar el frontend (30 min)

```
cd /opt/yaneth-sistema/Frontend
echo "VITE_API_URL=https://<dominio-del-sistema>" > .env.production
npm ci
npm run build
sudo mkdir -p /var/www/yaneth && sudo cp -r dist/* /var/www/yaneth/
```

2.7 Configurar Nginx y HTTPS (30 min)

Nginx entrega los archivos del cliente y reenvía /api y /health a la API local. Como el cliente usa rutas del navegador (React Router), cualquier ruta interna debe devolver index.html; de lo contrario, recargar una página da error 404.

```
sudo nano /etc/nginx/sites-available/yaneth

server {
    listen 80;
    server_name <dominio-del-sistema>;

    root /var/www/yaneth;
    index index.html;

    # API REST
    location /api/ {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 90s;   # la API espera hasta 60 s
    }

    # Verificación de estado (monitoreo)
    location = /health {
        proxy_pass http://127.0.0.1:3000;
    }

    # Archivos compilados con hash en el nombre: caché larga
    location /assets/ {
        expires 30d;
        add_header Cache-Control "public, immutable";
    }

    # Aplicación React: toda ruta interna devuelve index.html
    location / {
        try_files $uri $uri/ /index.html;
    }

    gzip on;
```

```
    gzip_types text/css application/javascript application/json image/svg+xml;
}

sudo ln -s /etc/nginx/sites-available/yaneth /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default
sudo nginx -t && sudo systemctl reload nginx

sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d <dominio-del-sistema> --redirect
sudo certbot renew --dry-run      # comprueba la renovación automática
```

Certbot agrega el bloque HTTPS (443) a la configuración, redirige HTTP a HTTPS y programa la renovación del certificado. Requiere que el dominio apunte a la IP pública y que el router reenvíe 80 y 443 hacia IP-APP. La documentación Swagger (/api-docs) no se publica por Nginx; para consultarla se abre un túnel SSH (ssh -L 3000:127.0.0.1:3000 usuario@IP-APP) y se visita http://localhost:3000/api-docs.

2.8 Prueba de conexión extremo a extremo (45 min)

```
curl http://127.0.0.1:3000/health               # directo a la API
curl https://<dominio-del-sistema>/health        # a través de Nginx
# ambos: {"ok":true,"message":"API funcionando"}
```

Después, abrir el dominio en un navegador, iniciar sesión con el administrador inicial y recargar (F5) una pantalla interna para confirmar la regla de index.html.

2.9 Explicación del entorno técnico al responsable del servidor (40 min)

Recorrido por la topología instalada, ubicación de los registros (logs) y reinicio de servicios (capítulo 6). Ajustes y margen para imprevistos: 55 min.

5.3 Día 3 – Configuración inicial de datos y pruebas

Tabla 9. Actividades del Día 3.

Actividad	Detalle	Tiempo
3.1 Cambiar la contraseña del administrador	Desde Usuarios → Contraseña, reemplazando la temporal.	10 min
3.2 Cargar catálogos maestros	Clientes, proveedores, vehículos, tipos de servicio y productos por tipo (Materia Prima, Producto Terminado, Insumo y Venta Directa). Cada Producto Terminado se crea con su plan de etapas.	45 min
3.3 Catálogos de producción	Tipos de etapa (Pelado, Corte, Fritura, Embalaje), tipos de merma y cuartos fríos. Se crean desde los formularios de Producción y Maduración ("+ Nuevo tipo…").	20 min
3.4 Estándar de maduración	Confirmar el umbral de Grados Brix (MATURATION_BRIX_THRESHOLD) y reiniciar la API si se cambió.	10 min
3.5 Alta de usuarios reales	Una cuenta por persona con su rol: Administrador, Producción, Logística o Piloto.	30 min

Actividad	Detalle	Tiempo
3.6 Capacitación breve	Registro correcto de la información propia de cada rol (Manual de usuario, Apéndice B).	45 min
3.7 Prueba de aceptación por módulo	Con datos reales: entrada de materia prima, control de maduración, proceso de producción con etapas, pedido, entrega y un reporte.	45 min
3.8 Tiempos de respuesta	Consultas y reportes en menos de 3 segundos en uso normal.	20 min
3.9 Validar respaldos	En el servidor de base de datos, ejecutar el respaldo manualmente y restaurarlo en una base de prueba (capítulo 7).	20 min
3.10 Aceptación final	Revisión con el responsable de la empresa y firma del Anexo A.	45 min
3.11 Ajustes finales	Atención de observaciones. Margen para imprevistos: 25 min.	45 min

6. Operación de los servicios

6.1 Orden de arranque

La API verifica al iniciar la conexión a la base de datos y a Redis; si alguno no responde, el proceso termina con el mensaje "No se pudo iniciar el servidor" y PM2 vuelve a intentarlo con una espera creciente. El orden de arranque es:

1. MySQL (servidor de base de datos)
2. Redis (servidor de colas)
3. API + worker (yaneth-api, servidor de aplicación)
4. Nginx (servidor de aplicación)

Con los servicios habilitados (systemctl enable y pm2 startup), este orden se respeta automáticamente al reiniciar un servidor único. En la variante B, tras reiniciar el servidor de base de datos o el de colas, se confirma que la API se recuperó con pm2 status y, si quedó detenida, se ejecuta pm2 restart yaneth-api.

6.2 Comandos de operación

Tabla 10. Comandos frecuentes de operación.

Acción	Comando (servidor donde se ejecuta)
Estado de la API	pm2 status (aplicación)
Reiniciar la API	pm2 restart yaneth-api (aplicación)
Detener / iniciar la API	pm2 stop yaneth-api / pm2 start yaneth-api (aplicación)
Ver registros de la API	pm2 logs yaneth-api --lines 100 (aplicación)
Probar y recargar Nginx	sudo nginx -t && sudo systemctl reload nginx (aplicación)
Estado de MySQL / Redis / Nginx	sudo systemctl status mysql | redis-server | nginx (en su servidor)
Reiniciar MySQL / Redis	sudo systemctl restart mysql | redis-server (en su servidor)

Acción	Comando (servidor donde se ejecuta)
Verificar la API	curl http://127.0.0.1:3000/health (aplicación)
Verificar Redis	redis-cli -h <IP-REDIS> -a '<contraseña-redis>' ping (aplicación)
Verificar MySQL	mysql -h <IP-DB> -u app_yaneth -p -e "SELECT 1;" (aplicación)

6.3 Registros (logs)

Componente	Dónde	Qué se registra
API	pm2 logs yaneth-api (~/.pm2/logs/), servidor de aplicación	Cada petición HTTP (morgan), errores 500 y arranque.
Nginx	/var/log/nginx/access.log y error.log, servidor de aplicación	Accesos, errores 502 (API detenida) y errores del servidor web.
MySQL	/var/log/mysql/error.log, servidor de base de datos	Arranque, apagado y errores del motor.
Redis	/var/log/redis/redis-server.log, servidor de colas	Arranque, autenticación y persistencia.
Negocio	Tabla auditoria_cambios	Quién cambió qué registro, cuándo y con qué valores.

Para evitar que los registros de PM2 llenen el disco se recomienda instalar la rotación:

```
pm2 install pm2-logrotate
```

7. Respaldos y restauración

7.1 Qué se respalda

- Base de datos (obligatorio). Contiene toda la información del negocio.
- Archivo Backend/.env y /etc/nginx/sites-available/yaneth. Contienen credenciales, el secreto JWT y la publicación del sistema; se guardan aparte, en un lugar seguro.
- Redis no requiere respaldo: solo contiene trabajos en tránsito.
- Código: está en los repositorios Git; basta con conocer la versión (commit) instalada.

7.2 Respaldo diario automático

El respaldo se programa en el servidor donde está MySQL (en la variante A, el servidor único) con un usuario de solo lectura. Se genera un volcado diario a las 02:00 con retención de siete días; el parámetro --single-transaction obtiene una copia consistente sin bloquear el uso del sistema.

```
sudo mysql
CREATE USER 'respaldo'@'localhost' IDENTIFIED BY '<contraseña-respaldo>';
GRANT SELECT, SHOW VIEW, TRIGGER, LOCK TABLES ON proyecto_graduacion.* TO
'respaldo'@'localhost';
EXIT;
```

```
sudo mkdir -p /respaldos && sudo chown $USER /respaldos
printf '[mysqldump]\nuser=respaldo\npassword=<contraseña-respaldo>\n' > ~/.my.cnf
&& chmod 600 ~/.my.cnf

crontab -e
# respaldo diario 02:00 y limpieza de copias con más de 7 días
0 2 * * * mysqldump --single-transaction --no-tablespaces proyecto_graduacion |
gzip > /respaldos/yaneth_$(date +\%F).sql.gz
30 2 * * * find /respaldos -name 'yaneth_*.sql.gz' -mtime +7 -delete
```

Al menos una vez por semana debe copiarse el respaldo más reciente fuera del servidor (disco externo u otro equipo de la red), por ejemplo desde otro equipo con scp usuario@IP-DB:/respaldos/yaneth_AAAA-MM-DD.sql.gz . para protegerlo ante una falla del equipo.

7.3 Restauración

1. Detener la API para que nadie registre datos durante la restauración: pm2 stop yaneth-api (servidor de aplicación).
2. Respaldar el estado actual, aunque esté dañado, por si se necesita revisarlo después.
3. Restaurar el archivo elegido en el servidor de base de datos:

```
gunzip -c /respaldos/yaneth_2026-10-21.sql.gz | sudo mysql proyecto_graduacion
```

4. Iniciar la API (pm2 start yaneth-api), comprobar /health e iniciar sesión.

7.4 Prueba periódica de restauración

Una vez al mes se restaura el último respaldo en una base de prueba del mismo servidor (por ejemplo proyecto_graduacion_prueba, creada con sudo mysql -e "CREATE DATABASE proyecto_graduacion_prueba;") y se comprueba el número de tablas y algunos registros recientes. Un respaldo que no se ha probado no garantiza la recuperación.

8. Mantenimiento

8.1 Rutinas de mantenimiento

Tabla 11. Plan de mantenimiento preventivo.

Frecuencia	Tarea
Diaria	Verificar que exista el respaldo del día. Revisar que la API esté en línea (pm2 status).
Semanal	Copiar el último respaldo fuera del servidor. Revisar errores en pm2 logs y en el log de Nginx. Revisar espacio en disco (df -h) en cada servidor.

Frecuencia	Tarea
Mensual	Prueba de restauración. Actualizaciones de seguridad del sistema operativo (apt upgrade) en cada servidor, en horario sin operación. Revisar usuarios activos y desactivar los del personal que ya no labora.
Trimestral	Revisar versiones de Node.js, MySQL, Redis y Nginx; confirmar que la renovación automática del certificado funciona (sudo certbot renew --dry-run).

8.2 Actualizar a una nueva versión del sistema

1. Avisar al personal y elegir un horario sin operación.
2. Hacer un respaldo manual de la base de datos (sección 7.2).
3. Descargar la nueva versión y revisar si incluye migraciones nuevas en sql/migrations/.
4. Aplicar las migraciones pendientes en orden de fecha (sección 8.3).
5. Actualizar dependencias y reiniciar la API.
6. Recompilar y publicar el frontend.
7. Verificar /health, iniciar sesión y recorrer los módulos principales.

```
cd /opt/yaneth-sistema/Backend && git pull && npm ci --omit=dev && pm2 restart
yaneth-api
cd /opt/yaneth-sistema/Frontend && git pull && npm ci && npm run build && sudo cp
-r dist/* /var/www/yaneth/
```

8.3 Migraciones de la base de datos

Cada cambio de estructura se guarda en sql/migrations/ con el formato AAAA-MM-DD_descripcion.sql. schema.sql ya contiene todos los cambios hasta su fecha de generación (9 de octubre de 2026); por ello, en una instalación nueva no se ejecutan migraciones. En una base existente se aplican, en orden, solo las migraciones posteriores a la última aplicada, desde el servidor de aplicación:

```
mysql -h <IP-DB> -u app_yaneth -p proyecto_graduacion < sql/migrations/2026-10-
09_add_codigo_trazabilidad.sql
```

Se recomienda llevar una bitácora con la fecha de aplicación de cada migración.

8.4 Gestión de usuarios y accesos

- Altas, cambios de rol, restablecimiento de contraseña y bajas se hacen desde el módulo Usuarios (rol Administrador).
- Un usuario dado de baja queda Inactivo: no puede iniciar sesión, pero sus registros históricos se conservan.
- Si se pierde el acceso de todos los administradores, se genera un nuevo hash y se actualiza directamente en la base:

```
UPDATE usuarios SET password_hash = '<hash-bcrypt>' WHERE username = 'admin';
```

8.5 Parámetros operativos

Parámetro	Dónde se cambia
Umbral de Grados Brix para "Listo para producción"	MATURATION_BRIX_THRESHOLD en .env y reinicio de la API.
Duración de la sesión	JWT_EXPIRES_IN en .env y reinicio de la API.
Dirección de MySQL o Redis (por ejemplo, al pasar de la variante A a la B)	DB_HOST / REDIS_HOST (y REDIS_PASSWORD) en .env y reinicio de la API; ajustar el usuario de MySQL y el firewall según la sección 5.1.
Tipos de etapa, tipos de merma y cuartos fríos	Desde la interfaz, en los formularios de Producción y Maduración.
Plan de etapas de cada producto terminado	Producción → pestaña Etapas por producto.
Stock mínimo y precio sugerido de cada producto	Módulo Productos.

8.6 Solución de problemas

Tabla 12. Problemas frecuentes y su solución.

Síntoma	Causa probable	Acción
La API no inicia: "No se pudo iniciar el servidor"	MySQL o Redis detenidos, o credenciales incorrectas en .env.	En su servidor: systemctl status mysql / redis-server. Desde la aplicación: probar con mysql -h y redis-cli -h; revisar DB_* y REDIS_*.
ECONNREFUSED o ETIMEDOUT hacia IP-DB o IP-REDIS	MySQL o Redis escuchan solo en 127.0.0.1, o el firewall del servidor remoto no tiene la regla para IP-APP.	Revisar bind-address (MySQL) o bind (Redis) y sudo ufw status en el servidor remoto.
Access denied for user 'app_yaneth'@'<IP>'	El usuario de MySQL se creó para otro origen o la contraseña no coincide.	Crear el usuario para 'app_yaneth'@'<IP-APP>' (actividad 1.2) o corregir DB_PASSWORD.
NOAUTH Authentication required / WRONGPASS	Falta REDIS_PASSWORD o no coincide con requirepass.	Corregir .env y ejecutar pm2 restart yaneth-api.
Error 502 Bad Gateway en el navegador	Nginx no alcanza la API: está detenida o no escucha en el puerto 3000.	pm2 status y pm2 logs; curl http://127.0.0.1:3000/health.
Error 404 al recargar una pantalla interna	Falta la regla try_files … /index.html en Nginx.	Revisar el bloque location / (sección 5.2, actividad 2.7).
Las pantallas cargan pero las operaciones tardan ~60 s y fallan	Redis no responde o el worker se detuvo.	redis-cli -h <IP-REDIS> ping; pm2 restart yaneth-api; revisar pm2 logs.
"Token invalido o expirado" / regreso al inicio de sesión	La sesión venció (JWT_EXPIRES_IN).	Iniciar sesión de nuevo.
Error 403 al usar un módulo	El rol del usuario no tiene acceso a esa ruta.	Revisar el rol en Usuarios.

Síntoma	Causa probable	Acción
El navegador no se conecta a la API	VITE_API_URL incorrecta, proxy de Nginx o firewall.	Revisar .env.production, recompilar el frontend y probar curl https://<dominio>/health.
MySQL no arranca tras un corte de energía	Apagado brusco que dañó archivos del motor.	Revisar el log de errores de MySQL; si no se recupera, restaurar el último respaldo (capítulo 7). Se recomienda UPS para los servidores.
Disco lleno	Respaldos o registros acumulados.	Revisar /respaldos, la retención y pm2-logrotate.

Los mensajes de validación del negocio (por ejemplo "El peso no cuadra" o "Faltan etapas requeridas") no son fallas técnicas: se explican en el Manual de usuario.

8.7 Buenas prácticas de seguridad

- Usar un JWT_SECRET propio, largo y aleatorio; si se sospecha que se filtró, cambiarlo (todas las sesiones se cierran).
- Publicar el sistema solo por HTTPS.
- No exponer los puertos 3000, 3306 ni 6379 a Internet: el 3000 es solo local y, en la variante B, 3306 y 6379 aceptan únicamente al servidor de aplicación.
- Configurar contraseña en Redis (requirepass) y comunicar los servidores por red privada, porque las conexiones hacia MySQL y Redis no van cifradas.
- Limitar SSH a la red interna y preferir llaves SSH a contraseñas.
- Contraseñas individuales para cada persona: no compartir cuentas.
- Mantener el archivo .env fuera de cualquier repositorio y con permisos restringidos (chmod 600).

Anexo A. Checklist de aceptación de la instalación

Marcar cada actividad al completarse y firmar al finalizar la instalación.

- ☐ Variante de despliegue definida (A o B) e IP privada fija en cada servidor
- ☐ Servidores con Ubuntu Server 24.04 actualizados
- ☐ MySQL y Redis instalados, con contraseña y escuchando solo donde corresponde
- ☐ Node.js 22, PM2 y Nginx instalados en el servidor de aplicación
- ☐ Firewall configurado en cada servidor; 3000, 3306 y 6379 sin acceso desde Internet
- ☐ Conexión a MySQL y Redis probada desde el servidor de aplicación
- ☐ Esquema creado (34 tablas) y administrador inicial registrado
- ☐ Variables de entorno configuradas con credenciales reales y JWT_SECRET propio
- ☐ API corriendo con PM2 y arranque automático
- ☐ Frontend publicado por Nginx con HTTPS y renovación automática del certificado

- ☐ Respaldo automático configurado y restauración probada
- ☐ Catálogos maestros y de producción cargados
- ☐ Usuarios reales dados de alta con su rol
- ☐ Prueba de aceptación por módulo sin observaciones abiertas
- ☐ Tiempos de respuesta verificados (< 3 s)

Firma de conformidad: _____ Fecha: _____

Anexo B. Instalación local para desarrollo

Para trabajar en el código en una computadora personal (Windows, Linux o macOS) no se necesitan Nginx ni PM2: el frontend corre con el servidor de desarrollo de Vite y la API con nodemon, que la reinicia al guardar cambios.

B.1 Requisitos

- Node.js 22 LTS (mínimo 20.19; Vite 8 no funciona con Node 18) y Git.
- MySQL 8 o MariaDB 10.6+. En Windows puede usarse XAMPP.
- Redis 7. En Windows se recomienda un contenedor de Docker Desktop; en Linux, el paquete redis-server.

B.2 Base de datos y Redis

```
mysql -u root -p -e "CREATE DATABASE proyecto_graduacion CHARACTER SET utf8mb4
COLLATE utf8mb4_general_ci;"
mysql -u root -p proyecto_graduacion < sql/schema.sql

docker run -d --name backend-redis -p 6379:6379 redis:7
```

El administrador inicial se crea igual que en producción: hash bcrypt de una contraseña e INSERT en la tabla usuarios.

B.3 Backend

```
git clone https://github.com/DecosUniversity/distri_yaneth_backend.git Backend
cd Backend
npm install
cp .env.example .env          # en Windows (cmd): copy .env.example .env
# ajustar DB_USER, DB_PASSWORD, DB_NAME=proyecto_graduacion y JWT_SECRET
npm run dev                   # http://localhost:3000/health y /api-docs
```

B.4 Frontend

```
git clone https://github.com/DecosUniversity/distri_yaneth_frontend.git Frontend
cd Frontend
npm install
cp .env.example .env.local    # VITE_API_URL=http://localhost:3000
npm run dev                   # http://localhost:5173
```

Vite escucha en todas las interfaces (puerto 5173), por lo que la interfaz puede probarse desde un teléfono de la misma red con http://<IP-del-equipo>:5173. En ese caso VITE_API_URL debe ser http://<IP-del-equipo>:3000, porque localhost apuntaría al propio teléfono.

B.5 Diferencias con producción

Aspecto	Desarrollo	Producción
Frontend	Servidor de Vite (npm run dev), recarga en caliente	Archivos compilados (npm run build) servidos por Nginx
API	nodemon (npm run dev)	PM2 (pm2 start src/server.js)
Acceso a la API	El navegador llama directamente a http://localhost:3000	Mismo dominio; Nginx reenvía /api
MySQL / Redis	Locales, usuario root, Redis sin contraseña	Usuario app_yaneth, Redis con contraseña, firewall
Variables	Backend/.env y Frontend/.env.local	Backend/.env (chmod 600) y Frontend/.env.production